Программная инженерия

Лекция 1: Введение в программную инженерию

Денисов Артём Владимирович

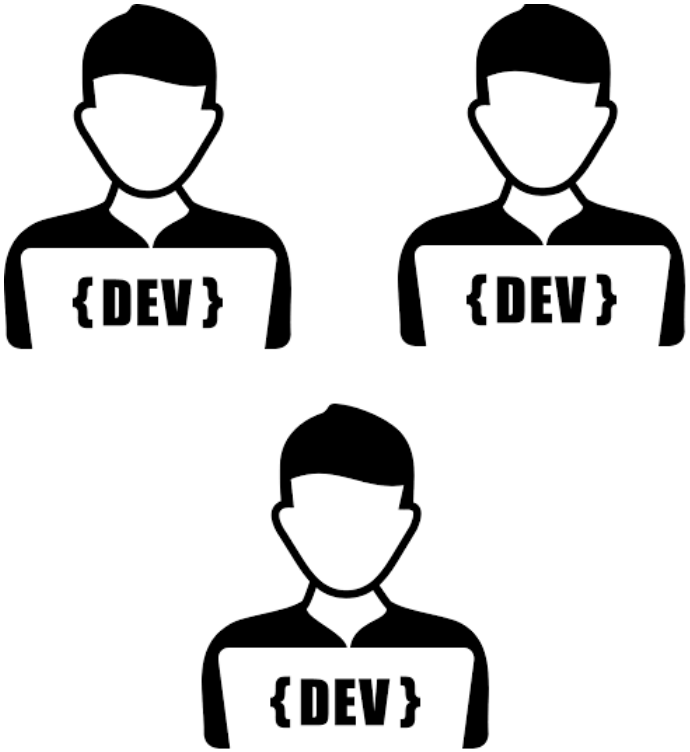
Уральский Федеральный Университет

2026

Как сейчас разрабатывается ПО?

Один разработчик

Проблема 1. Появилась команда

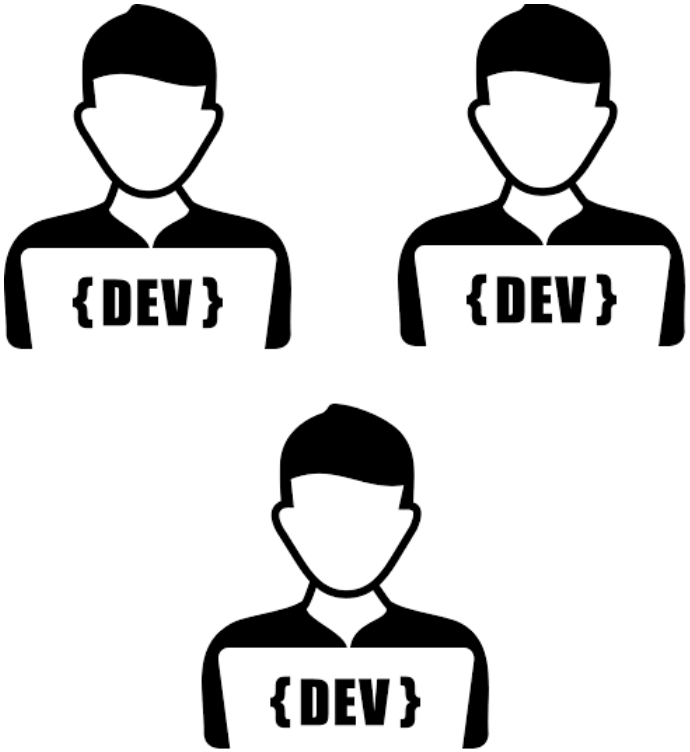


Frontend, Backend, Tester, DevOps, PM, ...

Проблемы

- Различное понимание требований
- Конфликты кода
- Различный стиль

Проблема 1. Появилась команда



Frontend, Backend, Tester, DevOps, PM, ...

Проблемы

- Различное понимание требований
- Конфликты кода
- Различный стиль

Решение

- Управление требованиями
- Система контроля версий, флоу ветвления

Проблема 2: Проект становится сложнее

Больше кода - больше проблем

Проблемы

- Плохие архитектурные решения
- Технический долг
- Изменения внедряются сложнее

Проблема 2: Проект становится сложнее

Больше кода - больше проблем

Проблемы

- Плохие архитектурные решения
- Технический долг
- Изменения внедряются сложнее

Решение

- Проектирование и архитектура
- Методологии управления техническим долгом

Проблема 3: Деплой

Проблемы

- Различное окружение
- Разные ОС
- Нагрузка на систему в продакшене
- Безопасность

Проблема 3: Деплой

Проблемы

- Различное окружение
- Разные ОС
- Нагрузка на систему в продакшене
- Безопасность

Решение

- Виртуализация, контейнеризация
- Распределение нагрузки

Проблема 4: Качество кода и надёжность программы

Проблемы

- Баги в продакшене
- Длительная ручная проверка каждой версии

Проблема 4: Качество кода и надёжность программы

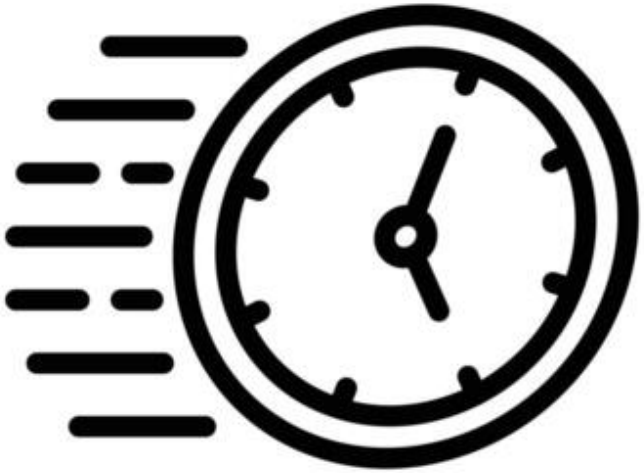
Проблемы

- Баги в продакшене
- Длительная ручная проверка каждой версии

Решение

- Тесты, тесты, тесты
- Code review
- Метрики качества

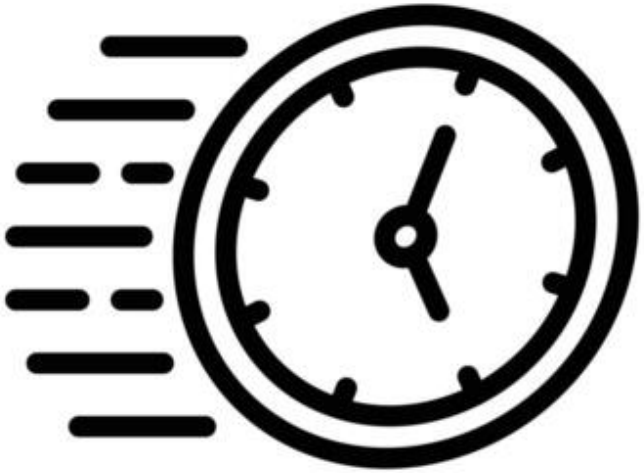
Проблема 5: Управление сроками и процессами команды



Проблемы

- Ручной деплой - долго и сложно
- Неточная оценка сложности задач

Проблема 5: Управление сроками и процессами команды



Проблемы

- Ручной деплой - долго и сложно
- Неточная оценка сложности задач

Решение

- Continuous Delivery, Continuous Deployment
- Гибкие методологии разработки

Проблема 6: Сопровождение ПО

Проблемы

- Сложно поддерживать
- Сложно следить за ошибками, производительностью

Проблема 6: Сопровождение ПО

Проблемы

- Сложно поддерживать
- Сложно следить за ошибками, производительностью

Решение

- Мониторинг, логирование
- Grafana, Prometheus, Loki, ...

Как правильно?

Просто написать код недостаточно

Что такое программная инженерия?

Точное определение

Программная инженерия (Software Engineering) — это инженерная дисциплина, применяющая системный и измеримый подход к созданию, эксплуатации и сопровождению качественного программного обеспечения

Что же такое программная инженерия?

- **Инженерная дисциплина** — применение проверенных методов, стандартов и практик
- **Системный подход** — охват всего жизненного цикла
- **Измеримый подход** — контроль на основе метрик и данных

Программирование — это код.
Программная инженерия — это продукт.

Появление программной инженерии

Разработка ПО для программы «Аполлон», Маргарет Гамильтон впервые использовала термин «программная инженерия», 1960-е

«Когда я впервые придумала термин, никто о нём раньше не слышал, по крайней мере, в нашем мире. Долгое время это была постоянная шутка. Им нравилось подшучивать над моими радикальными идеями.
Это был памятный день, когда один из самых уважаемых гуру аппаратного обеспечения объяснил всем на встрече, что он согласен со мной, что процесс создания программного обеспечения также следует рассматривать как инженерную дисциплину, как и в случае с аппаратным обеспечением.»

Маргарет и расчеты для программы «Апполона»

Идеи Гамильтон

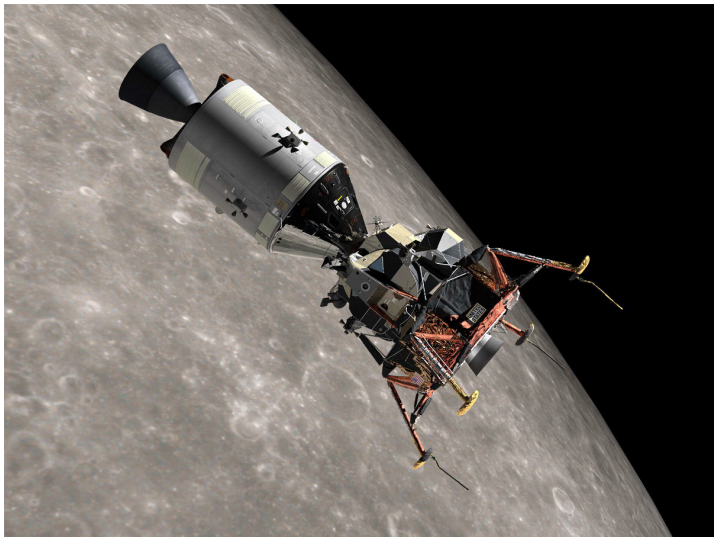
- **Идея поиска ошибки** - заключалась не в том, чтобы спланировать и сделать все правильно с первого раза, а в том, чтобы иметь возможность рассмотреть все варианты, когда что-то пошло не так.
- **Идея безопасного сбоя** - мы никогда не сможем закодировать каждый сценарий, надо писать код так, что наша система справляется с неожиданными сбоями и продолжает успешно работать

Пример: «Аполлон-11»

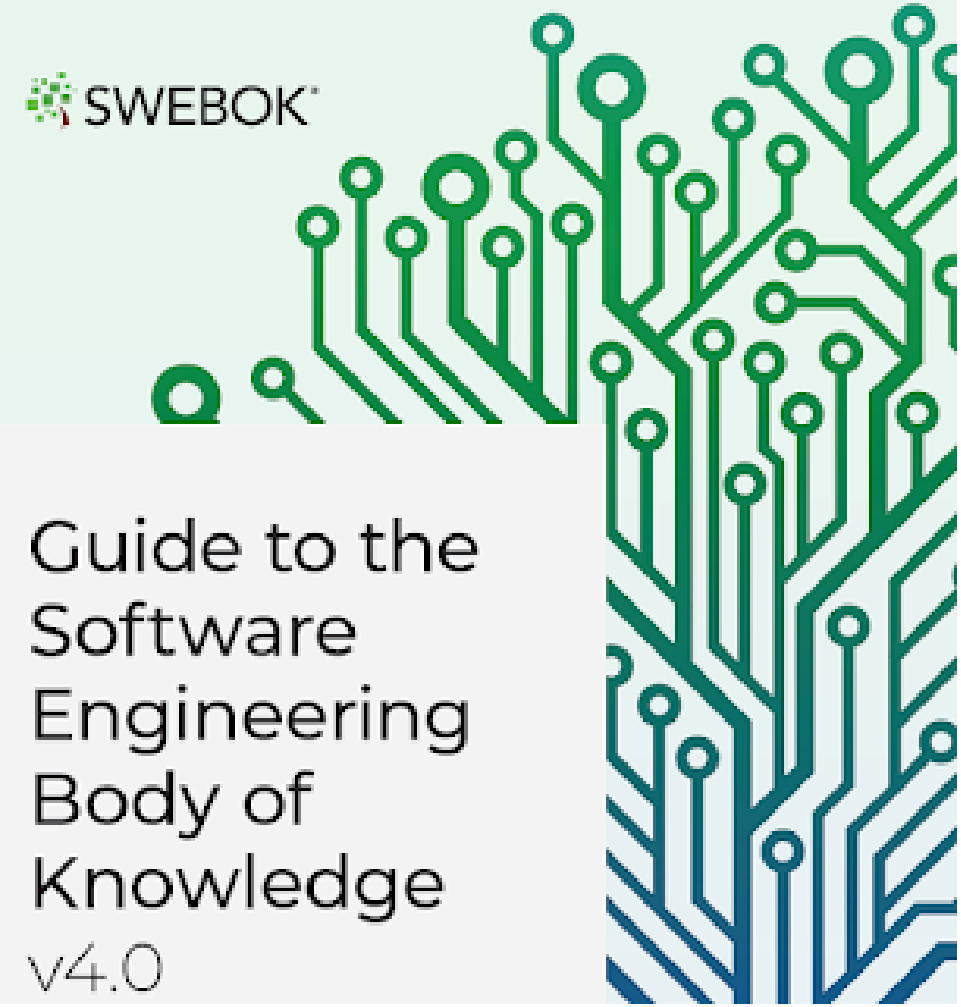
20 июля 1969

За 3 минуты до посадки компьютер выдал аварийные сигналы 1201, 1202 — перегрузка системы.

Код Гамильтон автоматически распознал проблему, отключил второстепенные задачи и сконцентрировался на управлении посадкой.



ПИ - инженерная дисциплина



SWEBOK

Что такое SWEBOK?

Software Engineering Body of Knowledge — международный стандарт профессиональных знаний в разработке ПО

- Публикуется **IEEE Computer Society**
- Признан стандартом **ISO/IEC TR 19759**
- Не учебник, а справочник профессиональных знаний
- Отвечает на вопрос: «Что именно входит в дисциплину программной инженерии?»

Зачем нужен SWEBOK?

Функции SWEBOK

1. **Определяет границы профессии**
   - Что относится к SE, а что к CS, системной инженерии, менеджменту
2. **Создаёт единый язык**
   - Разработчики из разных стран говорят об одном и том же
3. **Основа для образования**
   - Учебные программы и сертификация специалистов
4. **Карьерное развитие**
   - Roadmap от джуниора до архитектора

SWEBOK v4.0 — Октябрь 2024

Области знаний SWEBOK V4.0

1.	Software Requirements
2.	Software Architecture *
3.	Software Design
4.	Software Construction
5.	Software Testing
6.	Software Engineering Operations *
7.	Software Maintenance
8.	Software Configuration Management
9.	Software Engineering Management
10.	Software Engineering Process
11.	Software Engineering Models and Methods
12.	Software Quality
13.	Software Security *
14.	Software Engineering Professional Practice
15.	Software Engineering Economics
16.	Computing Foundations
17.	Mathematical Foundations
18.	Engineering Foundations

* — новые отрасли знаний для v4

Добавлены 3 ключевые области

18 областей знаний SWEBOK v4

Основные процессы разработки (1–6)

1. **Software Requirements** — работа с требованиями
2. **Software Architecture** — NEW: архитектурное проектирование
3. **Software Design** — детальное проектирование компонентов
4. **Software Construction** — кодирование и сборка
5. **Software Testing** — тестирование на всех уровнях
6. **Software Engineering Operations** — NEW: эксплуатация и DevOps

18 областей знаний SWEBOK v4

Поддержка и управление (7–10)

7. **Software Maintenance** — сопровождение и эволюция систем
8. **Software Configuration Management** — версии и конфигурации
9. **Software Engineering Management** — управление проектами
10. **Software Engineering Process** — процессы жизненного цикла

18 областей знаний SWEBOK v4

Качество и безопасность (11–13)

11. **Software Engineering Models and Methods** — модели и методологии
12. **Software Quality** — обеспечение качества
13. **Software Security** — NEW: безопасность ПО

18 областей знаний SWEBOK v4

Профессиональные и фундаментальные знания (14–18)

14. **Software Engineering Professional Practice** — этика и практика
15. **Software Engineering Economics** — экономика разработки ПО
16. **Computing Foundations** — основы компьютерных наук
17. **Mathematical Foundations** — математические основы
18. **Engineering Foundations** — инженерные основы

18 областей = полная карта профессиональных знаний

Что SWEBOK НЕ делает?

Важно понимать ограничения

SWEBOK — это...

- **Не учебник** — нет пошаговых инструкций «как написать код»
- **Не инструмент выбора технологий** — не скажет, React или Vue выбрать
- **Не методология** — документирует разные подходы, не навязывает конкретный

SWEBOK — это...

Справочник профессиональных знаний, а не руководство к действию

SWEBOK v4.0

Актуальный срез профессии

Состояние программной инженерии по состоянию на октябрь 2024

Доступ

`https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf`

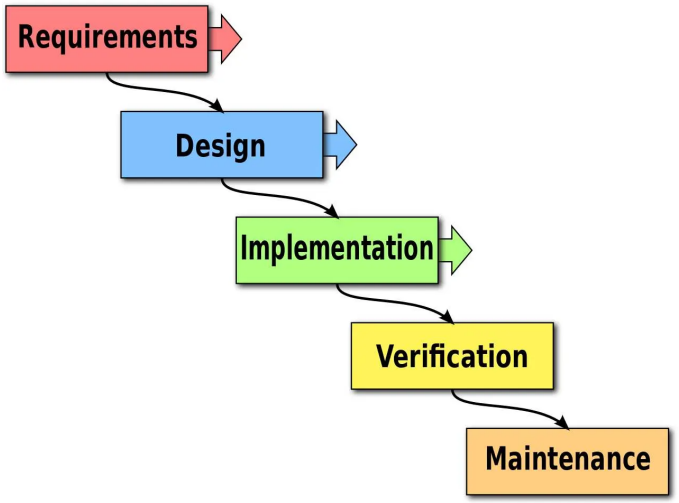
411 страниц

Эволюция жизненного цикла ПО

Жизненный цикл ПО

- Структурированный процесс создания, запуска и поддержки ПО.
- Цель: качество, управляемые риски, предсказуемые сроки и бюджет.
- Путь от идеи до вывода системы из эксплуатации.
- Эволюция моделей: Waterfall $\rightarrow$ итеративные $\rightarrow$ Agile $\rightarrow$ DevOps-подходы.

Waterfall: первый шаг к структуре



- Линейная последовательность этапов, без возвратов.
- Сильный упор на upfront-планирование и документацию.
- Хорошо для стабильных требований и жёстко регулируемых доменов.
- В остальных случаях - плохо.

Waterfall: этапы

1. Требования
2. Архитектура (high-level design)
3. Детальное проектирование
4. Реализация
5. Тестирование
6. Внедрение
7. Поддержка

Waterfall: плюсы/минусы

Преимущества:

- Простая и понятная структура
- Прогнозируемость этапов
- Высокая трассируемость
- Хорошая документация

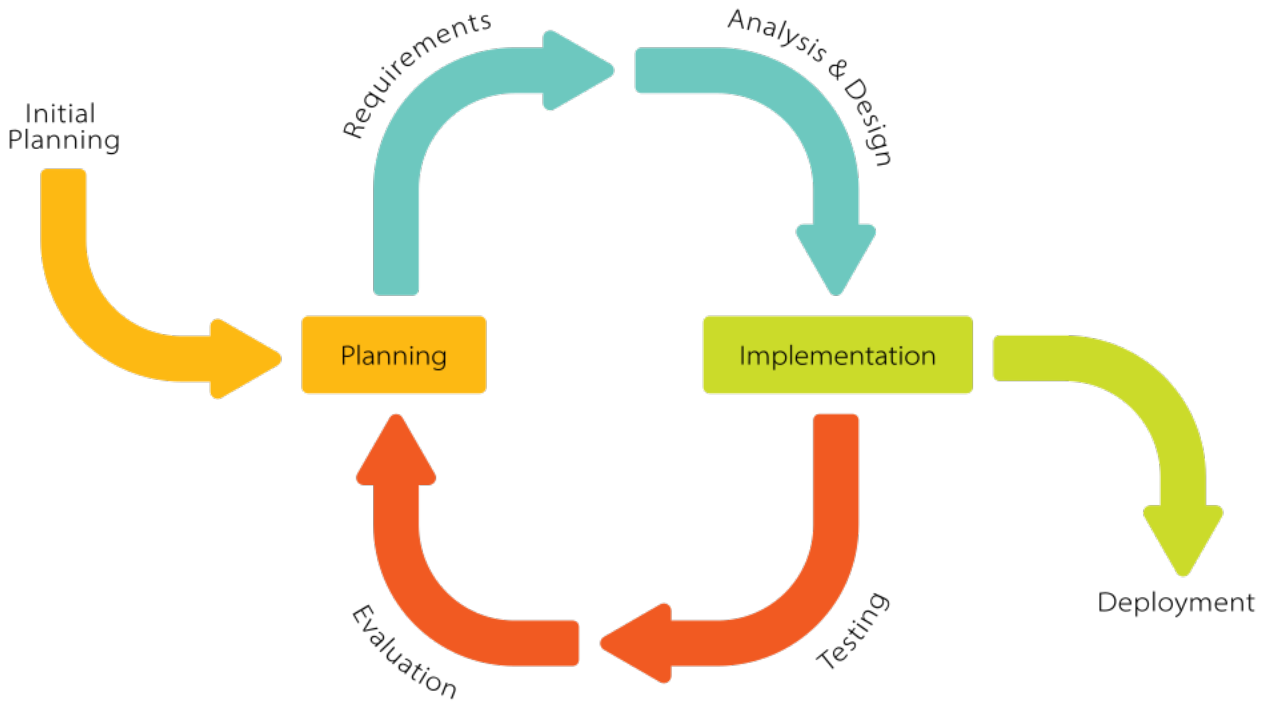
Недостатки:

- Позднее выявление проблем
- Дорогие изменения при смене требований
- Низкая гибкость

Почему ищем альтернативы

- Реальный фидбек от пользователей приходит слишком поздно
- Рынок и требования меняются быстрее, чем «успевает» Waterfall

Итеративные модели: шаг к гибкости



- Разработка циклами: анализ $\rightarrow$ дизайн $\rightarrow$ код $\rightarrow$ тест
- Каждый цикл даёт инкремент продукта
- Ранний фидбек и возможность корректировать курс

Инкрементальная и спиральная

Инкрементальная модель

- Продукт собирается по частям (инкрементам)
- Каждый инкремент — рабочая версия для пользователя

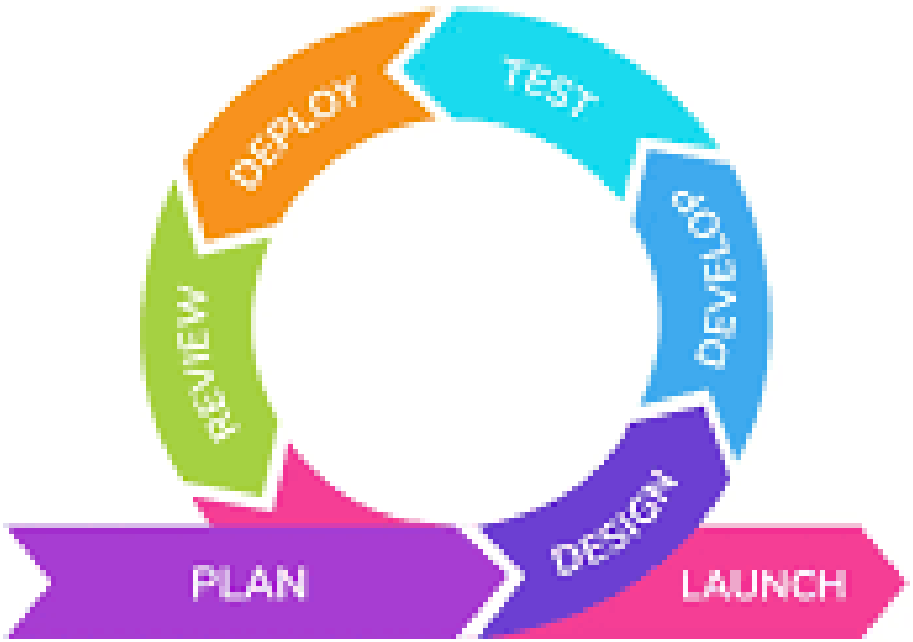
Спиральная модель

- Циклы по спирали: планирование, анализ рисков, разработка, оценка
- На каждом витке уточняются требования и решения

Ограничение

- Даёт итерации и фидбек, но ещё не задаёт общей культуры и практик команды.

Agile: ответ на жёсткость



- Итеративно-инкрементальная разработка короткими циклами.
- Фокус: люди, фидбек, работающий продукт, изменения.

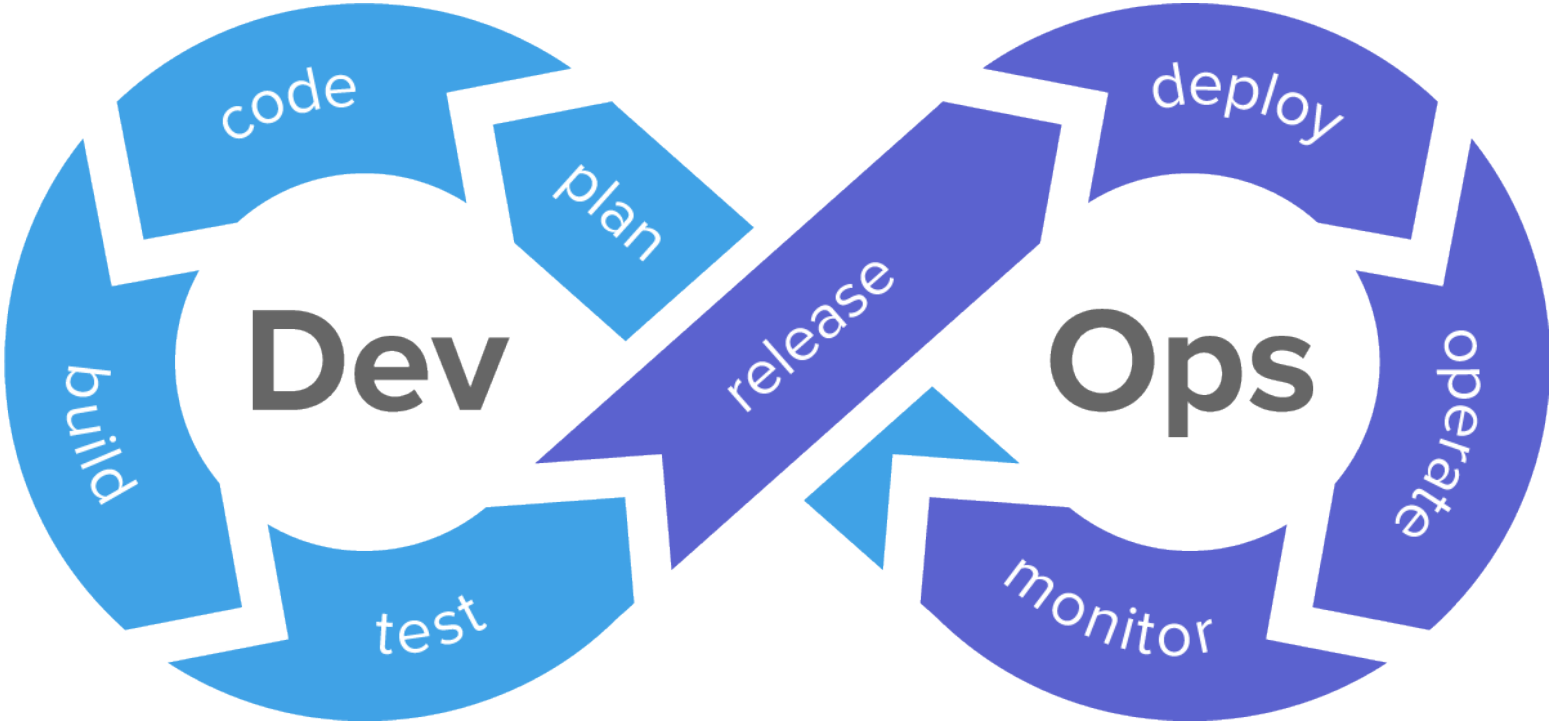
Agile: когда полезен

- Быстро меняющийся рынок и требования.
- Стартапы, веб/мобильные B2C-сервисы.
- Команды, готовые к самоорганизации и плотному общению с заказчиком.

Новая проблема

- Разработка ускорилась, но деплой и эксплуатация часто остаются «узким местом».

DevOps: расширение цикла до продакшена



- Объединение Development и Operations для непрерывной поставки
- CALMS: Culture, Automation, Lean, Measurement, Sharing
- Цели: скорость, надёжность, предсказуемость релизов

DevOps: ключевые практики

- CI: частые коммиты, авто-сборки и тесты
- CD/Continuous Deployment: автоматизированный путь до продакшена
- IaC: инфраструктура как код (Terraform, Ansible)
- Observability: логи, метрики, трассировки

Связь с Agile

- Agile ускоряет изменения, DevOps делает частые изменения безопасными.

DevOps: эффекты и ограничения

- Больше релизов, быстрее восстановление.
- Выше устойчивость сервисов.
- Высокий порог входа: инструменты, культура.
- Трудно внедрять в тяжёлом легаси без рефакторинга.

Итог эволюции

1. Waterfall структурировал разработку
2. Итеративные модели добавили циклы
3. Agile — гибкость управления
4. DevOps внедрил непрерывную и безопасную поставку до продакшена

Источники изображений

1. https://uxwing.com/programmer-icon/
2. https://ru.pngtree.com/freepng/minimalist-business-growth-chart-icon_21092238.html
3. https://ru.pinterest.com/pin/programmer-tshirt-it-works-on-my-machine-by-originto--5770305765691941/
4. https://pngtree.com/freepng/computer-crash-error-failure-system-flat-color-icon-vector_4963619.html
5. https://www.vecteezy.com/free-vector/fast-clock
6. https://www.flaticon.com/ru/free-icon/technical-support_5466151
7. https://monday.com/blog/project-management/waterfall-methodology/
8. https://en.wikipedia.org/wiki/Iterative_and_incremental_development
9. https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcRceeyRNLGISKcnfPN8RH38ZVg_wybD9M8RhQ&s